

Fiche de procédure Docker

1. Qu'est ce que Docker ?

Docker est une plateforme de virtualisation légère qui permet de créer, déployer et exécuter des applications de manière isolée et portable.

Elle regroupe chaque application et ses dépendances (bibliothèques, outils système, code et configuration) dans une unité standardisée appelée conteneur, garantissant ainsi un fonctionnement identique quel que soit l'environnement d'exécution.

2. Prérequis avant d'installer Docker

Avant toute installation, il est conseillé de supprimer d'anciens paquets susceptibles d'entrer en conflit avec la version officielle de Docker.

Les paquets non officiels à désinstaller sont :

- docker.io
- docker-compose
- docker-doc
- podman-docker

Pour ce faire :

```
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do  
  sudo apt-get remove -y $pkg  
done
```

Ensuite, il faut mettre à jour les dépôts et installer les dépendances :

```
sudo apt update  
sudo apt install -y ca-certificates curl
```

Puis il faut ensuite créer le dossier destiné à contenir la clé de signature Docker, puis télécharger et installer la clé GPG officielle :

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o  
/etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Et pour finir il faut ajouter le dépôt officiel Docker aux sources APT :

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \  
https://download.docker.com/linux/debian \  
$(. /etc/os-release && echo "${VERSION_CODENAME}") stable" \  
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null | sudo apt update
```

3. Installation de Docker

Installation de Docker et de ses composants principaux :

**sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin**

Le service de Docker démarre automatiquement après l'installation.

Une fois l'installation réussi il faut maintenant vérifier qu'il est bien actif :

sudo systemctl status docker

Certains systèmes peuvent avoir ce comportement désactivé et nécessitent un démarrage manuel.

Pour cela il faut faire : **sudo systemctl start docker**

Test de l'installation de Docker :

sudo docker run hello-world

Cette commande télécharge une image de test et l'exécute dans un conteneur. Lorsque le conteneur s'écoule, il imprime un message de confirmation et sort.

Si le message "Hello from Docker!" s'affiche, l'installation est réussie.

4. Commandes importantes de Docker

Commande	Explication
docker pull [nom de l'image]	Récupération d'une image depuis Docker Hub.
docker images	Liste toutes les images disponibles localement.
docker run -d -p [port Hôte]:[port Conteneur] --name [nom]	Exécuter un conteneur à partir d'une image. -d exécute le conteneur en arrière-plan. -p associe un port de la machine hôte à un port du conteneur. --name attribue un nom au conteneur pour le retrouver facilement.
docker ps -a	Liste de tous les conteneurs , y compris ceux arrêtés. -a signifie all.
docker ps	Liste de tous les conteneurs actifs .
docker exec -it [nom_du_conteneur] bash	Ouvrir un terminal à l'intérieur d'un conteneur en cours d'exécution.

docker stop [nom_du_conteneur]	Arrêter un conteneur.
docker start [nom_du_conteneur]	Démarrer un conteneur.
docker rm [nom_du_conteneur]	Supprimer un conteneur.
docker rmi [image]	Supprimer une image locale.

5. Création d'images personnalisées avec un Dockerfile

Docker permet de créer des images sur mesure à l'aide d'un fichier nommé *"Dockerfile"*. Ce fichier définit une base et les instructions nécessaires à la construction de l'environnement.

Exemple 1 : Une image "Hello World" en console

1 . Créer un fichier *Dockerfile* contenant :

```
FROM debian:latest
CMD ["echo", "Hello World!"]
```

2. Puis construire l'image à l'aide de la commande :

```
docker build -t [nom de l'image] .
```

Le point *"."* indique que le Dockerfile se trouve dans le répertoire courant.

3. Exécution de l'image :

```
docker run [nom_image]
```

Exemple 2 : Une image NGINX avec une page HTML personnalisée

1 . Créer un fichier index.html dans un répertoire de travail :

2. Puis créez un fichier Dockerfile contenant :

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

Cette instruction copie votre page dans le dossier par défaut de NGINX.

Explications :

On part de l'image officielle nginx:latest.

On copie notre page HTML dans le dossier /usr/share/nginx/html/ (là où NGINX cherche les fichiers par défaut).

3. Construire l'image à l'aide de la commande :

```
docker build -t [nom de l'image] .
```

4. Puis exécuter le conteneur :

```
docker run -d --name mon-nginx-container -p [port hôte]:80 [nom de l'image]
```

Explications :

-d : lancer en arrière-plan (daemon).

--name mon-nginx-container : donner un nom au conteneur pour le référencer plus facilement.

-p [port hôte]:80 : faire le lien entre le port 8080 de la machine hôte et le port 80 du conteneur (où NGINX écoute).

[nom de l'image] : nom de l'image que tu as construite.

En accédant à **http://[ip_machine] : [port hôte]**, la page HTML sera affichée depuis le conteneur NGINX.

6. Automatisation avec Docker Compose

Pour utiliser Docker Compose il faut :

1. D'abord créer un répertoire de travail.
2. Créer un fichier "*docker-compose.yml*".
3. Dedans il faudra mettre :

version: "La version"

services:

Nom du service:

build: .

image: Nom de l'image générée

ports:

- "[port hôte]:[port conteneur]"

4. Normalement les seuls fichiers existant sont :

- Le contenu qui sera dans le conteneur
- Le Dockerfile
- Le docker-compose.yml

5. Si c'est le cas on peut maintenant construire l'image :

docker compose build

6. Et lancer le conteneur :

docker compose up => au premier plan

docker compose up -d => en arrière plan (dans ce cas un CTRL + C ne fonctionnera pas. Il faudra faire : **docker compose stop**)



Il est possible de devoir installer Docker Compose manuellement :

sudo curl -L

"https://github.com/docker/compose/releases/latest/download/docker-compose-\$(uname -s)-\$(uname -m)" -o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose

Exemple : Déploiement d'une solution WordPress + MySQL

1. D'abord créer un répertoire de travail :

```
mkdir ~/wordpress-mysql  
cd ~/wordpress-mysql
```

2. Créer un fichier "*docker-compose.yml*" :

```
nano docker-compose.yml
```

3. Dedans il faudra mettre :

```
version: '3.8'
```

services:

wordpress:

```
image: wordpress:latest
```

ports:

```
- "8080:80"          # Port externe 8080 → interne 80
```

environment:

```
WORDPRESS_DB_HOST: db
```

```
WORDPRESS_DB_USER: wordpressuser
```

```
WORDPRESS_DB_PASSWORD: wordpresspass
```

```
WORDPRESS_DB_NAME: wordpressdb
```

volumes:

```
- wordpress_data:/var/www/html
```

depends_on:

```
- db
```

db:

```
image: mysql:5.7
```

```
restart: always
```

environment:

```
MYSQL_DATABASE: wordpressdb
```

```
MYSQL_USER: wordpressuser
```

```
MYSQL_PASSWORD: wordpresspass
```

```
MYSQL_ROOT_PASSWORD: rootpass
```

volumes:

```
- db_data:/var/lib/mysql
```

volumes:

wordpress_data:

db_data:

4. Téléchargement des images et déploiement :

```
docker compose pull
```

```
docker compose up
```

5. Ouvrir le navigateur à l'adresse :

```
http://[ IP de la VM ]:8080
```

7. Gestion et maintenance

Il est possible d'afficher les logs d'un conteneur pour cela il faut rentrer la commande :
docker logs [nom_du_conteneur]

Et pour un ensemble de service il faudra faire :

docker compose logs -f

Il est aussi possible de supprimer toutes les images et les conteneurs qui ne sont pas utilisés.

- Pour un image : **docker image prune -a**
- Pour un conteneur : **docker container prune**

 Ces commandes libèrent de l'espace disque, mais elles suppriment définitivement les ressources non utilisées.

8. Bonnes pratiques

- Toujours donner des noms explicites aux conteneurs et images.
- Avant de supprimer une ressource, vérifiez son état.
- Surveillez la consommation d'espace disque : Docker peut accumuler de nombreuses couches d'images.